An LLM-Based Agent-Oriented Approach for Automated Code Design Issue Localization

Anonymous Author(s)

ABSTRACT

Large Language Models (LLMs) have demonstrated remarkable code generation capabilities, but their potential for identifying design issues in evolving codebases remains largely unexplored. Existing LLM-based approaches struggle to capture design principles effectively and reason about complex code relationships due to limitations such as fixed token sizes, difficulty understanding complex program analysis representations, such as call graphs, and the tendency to hallucinate information, hindering their applicability in software design improvement.

To address that, we propose LocalizeAgent, a novel approach that combines: (1) multi-agent in-context learning, (2) natural language-based code analysis representation, and (3) context-aware prompting. Our multi-agent framework enables collaborative refactoring, where specialized agents work together to identify design issues, suggest targeted improvements, and rank the suggestions.

We evaluate LocalizeAgent on a diverse set of real-world codebases from a refactoring dataset. Our extensive evaluations demonstrate significant improvements in design issue localization accuracy over the baseline approaches. LocalizeAgent achieves a 138%, 166%, and 206% relative improvement in exact match accuracy for localizing issues with information hiding, complexity, and modular design. Further experiments also show the contributions of each component of our approach to the overall performance.

1 INTRODUCTION

Software design quality plays a crucial role in the long-term maintainability, extensibility, and evolution of a system. As codebases grow and evolve, design issues such as poor modularity, tight coupling, and excessive complexity can emerge, hindering developers' productivity and system quality. Addressing these design issues is essential to prevent the accumulation of technical debt.

While Large Language Models (LLMs) have demonstrated remarkable code generation capabilities [7, 23], their potential for recognizing design issues in evolving codebases remains largely unexplored. Current LLM-based approaches face several limitations that hinder their effectiveness in this domain. Fixed token sizes restrict the amount of context LLMs can process, making it challenging to understand large codebases holistically [15]. Moreover, complex structure-based representations of code, such as abstract syntax trees and control flow graphs, are difficult for LLMs to parse and reason about effectively [15]. Additionally, LLMs tend to generate false or inconsistent information, a phenomenon known as "hallucination" [15], which can lead to incorrect refactoring suggestions. These limitations impede the applicability of existing LLM-based techniques in software design improvement.

To understand the capability of LLMs in the context of software design improvement, we propose LocalizeAgent, a novel approach that leverages LLMs to localize design issues in code, focusing on local design issues that can be identified and resolved within a single class. Examples of local design issues include god classes (classes trying to do too much) and feature envy (methods being more interested in other classes than their own).

Our intuition is that by enhancing LLMs performance with *multi-agents*, *context-aware prompting*, and *an LLM-friendly representation of code context*, we can overcome the limitations of LLMs and enable more effective design issue localization. The key ideas behind our approach are as follows: (1) **Leverage multiple specialized agents:** To effectively identify design issues, we introduce a multi-agent learning framework where specialized agents collaborate to analyze different aspects of the codebase. Each agent focuses on specific tasks, such as analyzing code, extracting code contextual information, and dynamically building the prompt by using different program analysis tools. (2) **LLM-Friendly Code Context Representation:** In practice, developers often rely on Integrated Development Environment (IDE)-based program analysis tools and graph-based static analysis outputs, such as call graphs and dependency graphs, to identify design issues in their codebases. However, these complex representations are not well-suited for LLMs due to their token size limitations and the potential for hallucination when processing programming concepts from such structures [15]. LocalizeAgent addresses this challenge by introducing an LLM-friendly representation of code context that captures essential information about the codebase's structure and design quality, enabling LLMs to reason more effectively about design issues. (3) **Context-Aware Prompt Generation:** We introduce a prompting technique that generates tailored prompts based on the specific refactoring context and code analysis summaries. (4) **LLM-based Ranking:** We hypothesize that LLMs can be effective for ranking refactoring suggestions by learning from vast amounts of code and extracting insights about the relative importance of different design issues. Thus, we propose an LLM-based ranking agent combined with static analysis tools to prioritize localized design issues.

To evaluate LocalizeAgent, we conduct an experiment on a diverse set of real-world codebases from a refactoring dataset [1]. Our results demonstrate that our approach achieves a 206% relative improvement in accuracy for localizing modularity design issues, a 166% improvement for complexity issues, and a 138% improvement for information hiding issues in comparison with baseline LLM-based techniques when considering the Exact Match @ 1 (EM@1) metric. Furthermore, our approach demonstrates significant gains across different refactoring types, with the best models outperforming the baseline by an average of 75% to 184% on the EM@1 metric. These results showcase the potential of leveraging enhanced LLM techniques, such as our multi-agent learning framework and context-aware prompting, for design quality improvement.

In brief, the main contributions of this paper are as follows:

- A novel approach combining multi-agent learning to analyze codebases, extract relevant information, build prompts, and rank generated output.

```
...
static void initialiseAndInstallRepository(IConfigurationElement
        remoteRepositoryElem, RepositoryPlugin localRepo, MultiStatus
        status, IProgressMonitor monitor) {
    SubMonitor progress = SubMonitor.convert(monitor, 3);
    String implicitStr =
            remoteRepositoryElem.getAttribute("implicit");
    String repoName = remoteRepositoryElem.getAttribute("name");
    if (repoName == null)
        repoName = "<unknown>";
    if ("true".equalsIgnoreCase(implicitStr)) {
        try {
            RemoteRepository repo = (RemoteRepository)
              remoteRepositoryElem.createExecutableExtension("class");
            repo.initialise(progress.newChild(1));
            installRepository(repo, localRepo, status,
                  progress.newChild(2));
        } catch (CoreException e) {
            String message = MessageFormat.format("Failed to
                  initialise remote repository {0}.", repoName);
            if (status != null)
                status.add(new Status(IStatus.ERROR,
                      Plugin.PLUGIN_ID, 0, message, e));
            Plugin.logError(message, e);
        }
    } else {
        progress.worked(1);
    }
}...
```

(a) Code Before Refactoring

```
...
static void initialiseAndInstallRepository(RemoteRepository repo,
        String repoName, RepositoryPlugin localRepo, MultiStatus
        status, IProgressMonitor monitor) {
    SubMonitor progress = SubMonitor.convert(monitor, 3);
-   String implicitStr =
        remoteRepositoryElem.getAttribute("implicit");
-   String repoName = remoteRepositoryElem.getAttribute("name");
-   if (repoName == null)
-       repoName = "<unknown>";
-   if ("true".equalsIgnoreCase(implicitStr)) {
      try {
-           RemoteRepository repo = (RemoteRepository)
              remoteRepositoryElem.createExecutableExtension("class");
            repo.initialise(progress.newChild(1));
            installRepository(repo, localRepo, status,
                  progress.newChild(2));
      } catch (CoreException e) {
            String message = MessageFormat.format("Failed to
                  initialise remote repository {0}.", repoName);
            if (status != null)
                status.add(new Status(IStatus.ERROR,
                      Plugin.PLUGIN_ID, 0, message, e));
            Plugin.logError(message, e);
      }
-   } else {
-       progress.worked(1);
-   }
} ...
```

(b) Code After Refactoring

Figure 1: Comparison of Before and After Refactoring

- A natural language-based representation of program analysis outputs that enable LLMs to understand design principles more effectively than graph-based representations.
- A context-aware prompt generation method that produces targeted prompts based on refactoring contexts.
- Experiments on a set of real-world refactoring benchmarks demonstrate our tool's effectiveness and applicability.

Data Availability: A replication package for LocalizeAgent, including the source code and dataset, is available at [3].

2 BACKGROUND

2.1 Design Issues and Refactorings

As software evolves, its maintainability is affected by unaddressed technical debts. Issues such as poor modularity, tight coupling, and excessive complexity are all instances of design issues that can impede developer productivity. Development activities such as refactorings can resolve these design issues by rewriting sections of code to better ease later development. For example, developers may extract a new method from a body of code to reduce code duplication and lower the complexity of a system.

2.2 LLMs and Agents

Large language models (LLMs) have seen adoption in a wide variety of code intelligence tasks such as code translation [17], automated program repair [5], automated code review [9], comment generation [8], and fuzzing [24]. The interplay of potentially billions of learned parameters and ultra-large-scale training data contribute to LLMs' performance on downstream tasks. A typical application of code intelligence involves supplying a description of the task in natural language alongside input data. Particularly, recent works have equipped LLMs with "agents" that allow the LLM to gather context for itself through the use of provided tools [5, 10, 25]. Examples of previous tools provided as agents include those to explore the necessary codebase [5, 25], test and apply patches [5], and plan their goals [5]. Pomian et al. [19] propose techniques to generate and rank a specific type of refactoring activity. In this work, we aim to design agents that supplement LLMs with statically aggregated context to effectively resolve design issues.

3 MOTIVATION

Large Language Models (LLMs) have demonstrated a remarkable ability in code understanding and other code-related tasks. However, while LLMs can produce syntactically correct code, ensuring that this code adheres to established design principles and maintains high quality remains a challenge. The primary question is whether LLMs can also recognize design issues inherent in codebases.

Consider the example in Figure 1. We have a real-world Java class that violates the *separation of concerns* design principle. The class handles multiple responsibilities, such as error handling and progress monitoring, leading to increased complexity and reduced maintainability. As a result, a software quality evaluation tool named PMD [18] reports a higher cyclomatic complexity for the code before refactoring (Figure 1a). Localizing such design issues can be challenging for developers, as it requires identifying the specific parts of the code that need to change. LLMs have the potential to assist in this process by analyzing the code and suggesting appropriate refactorings. However, our experiments with LLMs (e.g., GPT-4o) using naive prompts reveal their limitations in effectively identifying and localizing design issues in (Figure 1a) [2].

Observation 1 [Limitations of Current LLM-based Approaches]. While LLMs have shown promise in generating code, their ability to recognize and fix design issues is limited. Naive prompting techniques often fail to capture the nuances of design principles and best practices, leading to generated code that may be syntactically correct but lacks adherence to established design guidelines.

Observation 2 [Challenges in Processing Complex Code Representations]. LLMs require contextual information to understand and reason about design issues in codebases effectively. However, the use of complex code representations, such as call graphs and dependency graphs, can pose challenges for LLMs due to their limited token capacity and potential for inconsistencies [15]. These limitations hinder LLMs' ability to make informed refactoring decisions based on the intricate relations/dependencies within a codebase.

Motivated by these observations, we propose LocalizeAgent that uses multiple LLMs and program-analysis-based agents to enhance LLMs' ability to recognize design issues.

Key Ideas. Our approach is designed with the following ideas:

Key Idea 1 [Multi-Agent Collaborative Refactoring]. Recognizing design issues and determining appropriate refactorings is a complex task that requires analyzing various aspects of the codebase. Thus, we introduce a multi-agent learning framework where specialized agents collaborate to suggest refactoring actions. In particular, we break down the problem into different subtasks, each handled by specialized agents designed to extract and pass information to identify design issues.

Key Idea 2 [LLM-Friendly Code Context Representation]. To enable LLMs to effectively reason about design issues, we propose a novel code context representation that is tailored to their strengths. Based on observation 2, traditional code representations, such as abstract syntax trees (ASTs) and control flow graphs (CFGs), are complex and can exceed the token limits of LLMs, hindering their ability to process and understand the codebase holistically. Thus, we propose lightweight analysis techniques that generate concise natural language summaries of code metrics and dependencies. These summaries capture essential information about the project's structure and design, making it understandable for LLMs.

Key Idea 3 [Context-Aware Prompt Generation]. The effectiveness of LLMs in localizing design issues heavily relies on the quality and relevance of the prompts provided to them. Generic or poorly constructed prompts often lead to suboptimal results, as they fail to capture the specific nuances and requirements of different refactoring scenarios. To address this limitation, we introduce a context-aware prompt generation technique that dynamically adapts the prompts based on the unique characteristics of each refactoring context. By leveraging the information obtained from our agents, we generate prompts that are tailored to the specific design issues and refactoring opportunities present in the codebase. For instance, as the above example requires a "parameterizing variable" refactoring, the LLM is prompted to identify both the variable and functions that need to be changed.

Key Idea 4 [LLM-based Ranking Agent]: While localizing design issues is crucial, prioritizing and ranking these suggestions is equally important. Without a proper ranking mechanism, developers may waste valuable time and resources addressing irrelevant or low-impact suggestions, hindering the overall refactoring process.

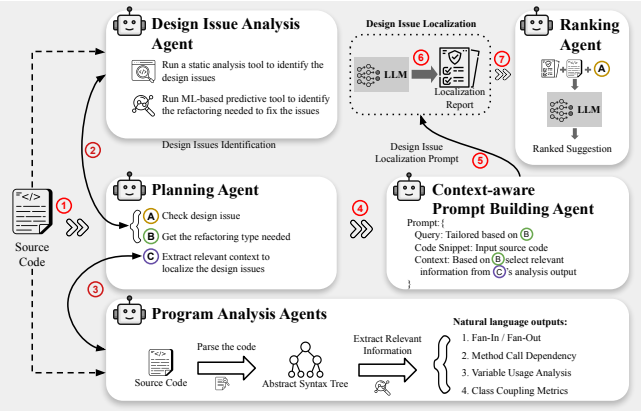


Figure 2: LocalizeAgent's Architecture Overview

This challenge is analogous to the well-studied problem of ranking code completion suggestions, where irrelevant or low-quality suggestions can be time-consuming and counterproductive for developers [12]. To tackle this challenge in the context of design issue localization, we introduce an LLM-based Ranking Agent that leverages the collective knowledge and reasoning capabilities of LLMs and static analysis tools to prioritize the localized design issues.

Problem Formulation. We formulate the problem of design issue localization using LLMs. Given a codebase $C_{\mathcal{B}}$ with potential design issues, our goal is to leverage LLMs to identify the specific functions or methods that should undergo refactoring to resolve these issues. By localizing the design issues at the function level, we aim to provide developers with actionable insights into which parts of the codebase require attention and improvement. Formally, let $C_{\mathcal{B}}$ be a codebase containing a set of functions $\mathcal{F} = f_1, f_2, \ldots, f_n$. Our objective is to identify a subset of functions $\mathcal{F}' \subseteq \mathcal{F}$ that exhibit design issues and which motivates a refactoring. The LLMs are tasked with analyzing the codebase $C_{\mathcal{B}}$ and generating a set of locations $\mathcal{L} = l_1, l_2, \ldots, l_m$, where each location l_i corresponds to a specific function $f_j \in \mathcal{F}'$ that needs refactoring.

4 APPROACH

4.1 Overview

Figure 2 illustrates our agent-oriented approach for localizing design issues in source code. It employs a cooperative communication paradigm, where agents work together towards the shared goal of identifying design issues. The communication structure is centered around the **Planning Agent** serving as the central authority that manages the information flow and delegates tasks to the other agents. The contents exchanged among the agents include source code and summaries of static analysis outputs as a natural language.

As depicted in Figure 2, the Planning Agent begins by invoking the **Design Issue Analysis Agent** to run a static analysis tool on the input source code, identifying potential design issues. Based on this information, the Planning Agent then triggers the **Program Analysis Agents** to extract relevant context by parsing the code into an AST and applying lightweight program analysis techniques. The agents generate concise natural language summaries capturing essential insights on the analysis' outputs regarding the code's

structure, dependencies, and design attributes. Next, the **Context-aware Prompt Building Agent** constructs tailored prompts based on the analysis summaries and the refactoring types determined by the Planning Agent. The generated prompts are fed into an LLM, which leverages them to localize the specific design issues within the codebase and outputs a refactoring suggestion. Finally, an LLM-based **Ranking Agent** takes the source code, the design issue, and suggestions to rank the most relevant suggestion higher.

4.2 Data Collection and Filtering Design Issues

4.2.1 ***Dataset.*** Our study uses refactoring datasets extracted from open-source code repositories [1]. RefactoringMiner [22] is used to collect the following information from the refactored code: (1) the type of refactoring applied and (2) the name of the refactored method. The details on the dataset will be discussed in § 8.1.1.

4.2.2 ***Rule-Based Design Issue Filtering.*** To focus our analysis on refactorings that address design issues, we employ PMD [18], a static analysis tool that detects potential design problems in source code. PMD provides a set of predefined rules that identify specific design issues. We categorize the detected issues into three main categories: modularity, information hiding, and complexity, each encompassing specific problems as defined in PMD's taxonomy.

Below, we present the list of PMD rules used in our study and their corresponding categories.

Modularity: `[God Class [18], Coupling Between Objects [18], Data Class [18]]`. These issues are detected under the modularity category, which includes classes performing too many functions, excessive coupling between objects, and classes serving as data stores.

Information Hiding: `[Law of Demeter [18], Excessive Parameter List [18]]`. These issues pertain to the excessive coupling of methods and classes and methods with too many parameters, both of which can compromise encapsulation and modularity.

Complexity: `[Too Many Methods [18], Excessive Public Count [18], Cyclomatic Complexity [18]]`. Complexity-related issues focus on classes with too many methods, excessive use of public methods and attributes, and high cyclomatic complexity that could hinder understandability and maintainability.

Based on the rules, we filter our dataset and ensure that the refactoring addresses genuine design improvements. To do so, let C be a commit representing a refactoring, and let C_{before} and C_{after} be the code snapshots before and after the refactoring, respectively. We denote the set of design issues detected by the PMD tool [18] in a code snapshot C_i as $I(C_i)$. Each issue $i \in I(C_i)$ belongs to one of the design categories $D = \{modularity, information_hiding, complexity\}$. We then apply the following criteria:

(1) Remove duplicate source code and code with extremely long lines to maintain dataset quality and tractability.
(2) For each commit C, run PMD on C_{before} and C_{after} to obtain the sets of design issues $I(C_{before})$ and $I(C_{after})$.
(3) Select commit C if and only if both:
   - $\exists i \in I(C_{before}) \wedge i \in D$: There exists at least one issue i in C_{before} that belongs to any of the design categories D.
   - $\forall i \in I(C_{before}) \cap D, i \notin I(C_{after})$: All design issues i present in C_{before} are resolved in C_{after}, i.e., the refactoring has fixed the identified design issues.

By applying these filtering criteria and leveraging PMD's rule-based design issue detection, we obtain a refined dataset $\mathcal{D}$ that contains refactorings addressing specific design issues. This filtered dataset $\mathcal{D}$ serves as the foundation for evaluating our agent-driven in-context learning approach to localize design issues.

5 PLANNING AGENT

The Planning Agent plays a central role in orchestrating the design issue localization process. The Planning Agent consists of two main components: the Design Issues Analysis Agent and the Program Analysis Agent, which work together to identify design issues and extract relevant context for localization.

5.1 Design Issues Analysis Agents

The Design Issues Analysis Agents are specialized entities within our approach tasked with identifying design issues and the refactoring needed for a given codebase.

First, the agent leverages static analysis tools, such as PMD [18], to scan codebases and detect potential design anomalies. PMD provides a set of predefined rules and heuristics that help identify issues related to poor modularity, lack of information hiding, and high complexity. For instance, PMD can detect coupling between objects and god classes, as shown in the example output below:

```
CouplingBetweenObjects: High amount of different
    objects as members denotes a high coupling

GodClass:  Possible God Class (WMC=78, ATFD=36,
    TCC=8.000%)
```

Second, the agent utilizes a ML-based tool to enhance the localization of issues by predicting the refactoring needs. We reuse the tool developed by Aniche *et al.* [1] on predicting the refactoring needs of a codebase. Given source code, the tools use a random forest algorithm to predict the refactoring type (i.e., move method, inline method, etc..). For instance, in the example in Figure 1, the code exhibits high cyclomatic complexity, and the ground truth indicates the need for variable parameterization to reduce complexity.

The information gathered from these analyses is then used in the context-aware prompt-building phase (Figure 2), ensuring that the prompts generated for the LLM are informed by detailed analysis, leading to more effective and precise suggestions.

5.2 Context Extraction via Program Analysis Agents

While the Design Issues Analysis Agent focuses on identifying design issues and determining refactoring strategies, the Program Analysis Agents are responsible for extracting relevant context from the codebase, aiming to aid the LLM in the issue localization.

Developers often rely on IDE-based tools or static analysis outputs like call graphs and dependency graphs to scan their codebases and identify potential design issues. However, directly feeding complex representations to LLMs can lead to hallucination and sub-optimal results due to their limited token capacity and inability

to process programming concepts from such representation effectively [11, 15]. Thus, we build an agent using lightweight analysis tools to extract relevant context for design issue localization.

5.2.1 Summary on Code Analysis. We propose a novel representation of code context more suitable for LLMs to address this challenge. Instead of using complex graph representations, we develop a set of lightweight analysis techniques that generate concise natural language summaries of key code metrics, dependencies, and design attributes. These summaries capture the essential insights needed for refactoring while being more easily digestible by LLMs.

The key novelty of our approach lies in parsing the AST to generate natural language summaries that serve as a context for the LLMs rather than directly feeding complex representations.

Let $C_{\mathcal{B}}$ be a codebase, and let $AST(C_{\mathcal{B}})$ be the abstract syntax tree representation of $C_{\mathcal{B}}$. We define a set of lightweight code analysis techniques $\mathcal{A} = A_1, A_2, \ldots, A_n$, where each technique A_i is a function that takes the AST as input and generates a summary S_i:

$$A_i : AST(C_{\mathcal{B}}) \rightarrow S_i \tag{1}$$

The summary S_i encapsulates the relevant code properties, relationships, and metrics captured by the analysis technique A_i. Formally, we define the summary generation process as follows:

$$\mathcal{S} = \{S_1, \ldots, S_n\}, \text{ where } S_i = A_i(AST(C_{\mathcal{B}})) \text{ for } i = 1, \ldots, n \tag{2}$$

The set $\mathcal{S}$ represents the collection of summaries generated by applying each lightweight code analysis technique A_i to the AST of the codebase $C_{\mathcal{B}}$.

5.2.2 Lightweight Code Analysis Tools. We perform the following:

Fan-in and Fan-out Analysis ($A_{FI/FO}$). This analysis computes the fan-in and fan-out metrics for each method in the codebase. Fan-in denotes the number of times a method is called, and fan-out denotes the number of methods that are being called within the concerned method [20]. Motivated by related works by Schwanke *et al.* and Layman *et al.* [14, 20], we hypothesize that including a summary of this analysis task can indicate the coupling and complexity issues of the system. This summary can effectively provide LLMs with potentially relevant information to localize future refactorings.

```
Total number of functions: 16
...
Function: initialiseAndInstallRepository
  Fanout (number of calls made by this function): 11
  Fanin (number of calls made to this function): 1
...
```

Call Relationships Analysis (A_{CR}). As opposed to fan-in and fan-out analysis, this analysis identifies the direct call dependencies between methods, helping to identify potential coupling and cohesion issues involving groups of specific methods. We speculate that methods with high interactions between themselves could be symptoms for future refactoring activity and that this summary can assist LLMs in localizing these.

```
...
Function: initialiseAndInstallRepository
 Dependencies: createExecutableExtension, initialise,
     convert, equalsIgnoreCase, logError, getAttribute,
     add, newChild, installRepository, format, worked
 Functions depending on initialiseAndInstallRepository:
     installImplicitRepositoryContents
 ...
```

Variable Usage Analysis (A_{VU}). This analysis tracks the scope and usage patterns of variables across different functions. We speculate that variables that are heavily used or are used as parameters across multiple functions could be motivation for future refactoring activity. Thus, organizing this information as input for an LLM can potentially increase performance.

```
...
Function: initialiseAndInstallRepository
    Variable: message
        Used as an argument in method invocation: logError
...
```

Class Coupling Analysis (A_{CC}). This analysis assesses the level of interdependencies between classes and methods, providing insights into the system's modularity and design quality at the class level. Thus, highly involved classes can become a code smell for future refactorings, and LLMs could leverage this presented information to better guide their localization performance.

```
Class: LocalRepositoryTasks
 Afferent Coupling (incoming dependencies): 0
 Efferent Coupling (outgoing dependencies): 44
```

6 CONTEXT-AWARE PROMPTING AGENT

The Context-aware Prompt Building Agent aims to build the prompt for the LLM in localizing the design issues. It is triggered by the Planning Agent once the Design Issues Analysis Agents have identified the design issues and the required refactoring types and the Program Analysis Agents have extracted the relevant code context. The Planning Agent passes the refactoring types and the code context summaries to the Context-aware Prompt Building Agent. Upon receiving this information the Context-aware Prompt Building Agent leverages it to generate targeted prompts for the LLM in order to localize the issues.

Formally, let $R = \{r_1, r_2, \ldots, r_n\}$ be the set of refactoring types that were identified by the Design Issue Analysis Agent.

Let $A = \{a_1, a_2, \ldots, a_m\}$ be the set of analysis tools, where each tool a_i provides a summary s_i of the code metrics and attributes.

For a given refactoring type $r_j \in R$, the context-aware prompting procedure identifies the relevant analysis summaries $S_{r_j} = \{s_i \mid a_i \text{ is relevant to } r_j\}$. It then generates a prompt p_{r_j} using a prompt generation function f:

$$p_{r_j} = f(r_j, S_{r_j}) \tag{3}$$

The prompt generation function f takes the refactoring type r_j and the relevant analysis summaries S_{r_j} as input and produces a natural language prompt p_{r_j} that guides the LLM in addressing the specific design issues associated with r_j.

Table 1: Analysis techniques used for each Design Issue

Design Issues	Analysis Used			
	$A_{FI/FO}$	A_{CR}	A_{VU}	A_{CC}
PV-Ref		✓	✓	
IM-Ref	✓	✓	✓	
IV-Ref			✓	
MM-Ref	✓	✓		✓

To illustrate this process, consider the *Parameterize Variable* refactoring type, denoted as r_p. Let $S_{r_p} = s_{VU}, s_{CR}$ be the relevant analysis summaries, where s_{VU} is the variable usage summary, and s_{CR} is the call relationship summary. The prompt generation function f would create a prompt p_{r_p}:

$$p_{r_p} = f(r_p, \{s_{VU}, s_{CR}\}) \tag{4}$$

As shown in Figure 3, the generated prompt p_{r_p} would ask the LLM to identify the function name that should be parameterized to reduce coupling. To aid the localization procedure, the prompt incorporates the contexts from the variable usage and call relationship summaries, guiding the LLM in the localization process.

Prompt Structure:

Query: { Analyze the code snippet to identify variables that should be parameterized to reduce coupling. Provide the variable name to parameterize, the function its in, and the function name that need to change. \n
},

Output Formatting: { Don't use ` or * on the response. Use the following format only: Function: <func_name> . Example: Function: get_user \n
},

Context: { { "Variable Usage Analysis": A_{VU} },
{"Class and Method Coupling Metrics": A_{CMC} }
}

Figure 3: Example of a prompt for issue localization

The context-aware prompt builder is performed for each refactoring type $r_j \in R$, generating a set of prompts $P = \{p_{r_1}, p_{r_2}, \ldots, p_{r_n}\}$ that cover the various design issues identified in the codebase.

To conduct an evaluation of our approach and assess its effectiveness, we carefully selected the refactoring types based on the filtering criteria described in § 4.2.2. After applying these criteria to the dataset provided by Aniche *et al.* [1], we identified four refactoring types that successfully resolved design issues and contained the necessary ground truth information:

- Parameterize Variable (PV-Ref): This refactoring extracts a variable as a parameter, reducing coupling and increasing flexibility.
- Inline Method (IM-Ref): Inline Method replaces a method call with the method's contents, eliminating the need for a separate method and potentially reducing complexity.
- Inline Variable (IV-Ref): This refactoring replaces variable references with the variable's initializer expression, simplifying the code structure.
- Move Method (MM-Ref): Move Method moves a method to another class where it is more closely related, improving cohesion and reducing coupling between classes.

Table 1 presents a mapping that shows which analysis techniques are used for each refactoring type (marked as ✓). This mapping is based on a careful examination of the characteristics and requirements of each refactoring type, ensuring that the most relevant analysis summaries are utilized in the prompt generation process.

7 DESIGN ISSUE LOCALIZATION AND RANKING

The In-Context Learning (ICL) procedure leverages the context-aware prompt that contains the analysis tools' outputs to guide the code refactoring suggestion process.

7.1 Design Issue Localization

Let $C_{\mathcal{B}}$ be the original codebase, and let $\mathcal{A} = \{a_1, a_2, \ldots, a_n\}$ be the set of lightweight code analysis techniques, where each technique a_i generates a summary S_i of the code metrics and attributes. Given a refactoring type r, the dynamic prompting system generates a prompt p_r based on the relevant analysis summaries $\mathcal{S}_r = \{S_i \mid A_i \text{ is relevant to } r\}$. The prompt p_r is then used to guide the learning model $\mathcal{M}$ in localizing the design issues related to the refactoring type r. The learning model $\mathcal{M}$ takes as input the original codebase $\mathcal{B}$, the prompt p_r, and the relevant analysis summaries $\mathcal{S}_r$. It generates a set of locations $\mathcal{L}_r = \{l_1, l_2, \ldots, l_k\}$ that correspond to the design issues identified by the prompt:

$$\mathcal{L}_r = \mathcal{M}(C_{\mathcal{B}}, p_r, \mathcal{S}_r) \tag{5}$$

Each location $l_i \in \mathcal{L}_r$ represents a specific function name within the codebase where a design issue related to the refactoring type r has been identified.

7.2 Ranking Localized Design Issues

After localizing the design issues, we perform a post-processing step to rank them based on their relevance and impact. Let $\mathcal{DI} = \{DI_1, DI_2, \ldots, DI_m\}$ be the set of design issues from PMD, and $\mathcal{L} = \{\mathcal{L}r_1, \mathcal{L}r_2, \ldots, \mathcal{L}_{r_m}\}$ be the set of localized design issues. The Ranking Agent $\mathcal{RA}$ prioritizes the localized design issues based on their relevance and impact.

Given the original codebase C_B, the set of localized design issues $\mathcal{L}$, and the code context summaries $\mathcal{S} = \{S_1, S_2, \ldots, S_n\}$ generated by the analysis techniques $\mathcal{A} = \{a_1, a_2, \ldots, a_n\}$, $\mathcal{RA}$ generates a ranking score ρ_i for each issue $l_i \in \mathcal{L}$:

$$\rho_i = \mathcal{RA}(C_B, l_i, \mathcal{S}, \mathcal{DI}) \tag{6}$$

The ranking score ρ_i is computed by the LLM-based Ranking Agent $\mathcal{RA}$, which assesses the relevance and impact of each localized function, l_i, considering the source code context C_B and the code context summaries $\mathcal{S}$, and the corresponding design issue DI_j.

The Ranking Agent $\mathcal{RA}$ then sorts the localized design issues based on their ranking scores, producing a prioritized list $\mathcal{L}_{sorted}$.

8 EVALUATION

For evaluation, we seek to answer the following research questions:

RQ1 (§ 8.2): How effective is LocalizeAgent on localizing design issues? In this RQ, we assess the ability of LocalizeAgent to localize design issues in source code. We consider two scenarios:

(A) **Localizing a single design issue:** We evaluate LocalizeAgent's effectiveness in identifying the specific location of a single design issue within a codebase.

(B) **Localizing multiple design issues:** We investigate LocalizeAgent's performance in localizing multiple design issues simultaneously, assessing its ability to handle complex scenarios with various design problems.

RQ2 (§ 8.3): How sensitive is LocalizeAgent under different API settings when suggesting design issues refactoring? In this RQ, we perform a sensitivity analysis to evaluate how LocalizeAgent's performance varies under different settings and parameters. This analysis helps us understand the robustness and stability of our approach in various scenarios.

RQ3 (§ 8.4): How does each component in LocalizeAgent contribute to the performance of localizing issues? We investigate the contribution of each key component in LocalizeAgent, namely the multi-agent learning framework, natural language-based code analysis representation, and context-aware prompting.

RQ4 (§ 8.4): What are the time complexity and cost of localizing design issues? In this RQ, we evaluate the efficiency of LocalizeAgent by measuring the time and the computational cost required to localize design issues in codebases of varying sizes.

Table 2: Statistics of the Evaluation Dataset

Refactoring Types	Design Issues							
	Modularity		Information Hiding		Complexity		Total	
	Single	Multi	Single	Multi	Single	Multi	Single	Multi
PV-Ref	18	1	9	12	11	5	38	18
IM-Ref	237	43	48	83	409	102	694	228
IV-Ref	66	11	21	31	100	24	187	66
MM-Ref	459	424	119	338	519	580	1097	1342
Total	780	479	197	464	1039	711	2016	1654

8.1 Experimental Setup

8.1.1 **Datasets**. We used the refactoring dataset released by Aniche *et al.* [1]. The dataset contains 8.8M commits that include refactoring across 20 different types. Each Java class in the dataset has instances of before and after refactoring, with labels indicating the refactoring types applied, the method name that changed, and its line number. To identify this information, the authors used a static analysis tool, RefactoringMiner [22]. Following a similar methodology as NatGen [6], we sampled 2 million Java classes, extracted the refactoring information (i.e., refactoring type and ground truth), and discarded code duplicates. In this work, we focus on code design issues. Specifically, we kept the datasets that satisfy the following conditions described in § 4.2.2.

Table 2 presents the statistics of our evaluation dataset. We categorize the dataset based on the number of refactoring operations performed to address design issues in each code instance. If only a single refactoring operation (e.g., one "Move Method") is applied to fix the design issues in a codebase, we consider it a single issue. Conversely, if multiple refactoring operations of the same type (e.g., more than one "Move Method") are applied to a single codebase to resolve design issues, we classify it as a multiple issue.

8.1.2 **Implementation**. Our implementation leverages GPT-4o, Claude 3, and Gemini 1.0 APIs to localize design issues. The lightweight static analysis techniques are implemented using Python and the JavaLang parser [13] to construct the AST and generate concise natural language summaries. PMD [18], a static analysis tool, is employed on a Linux environment to identify design issues and verify the effectiveness of refactorings. The generated summaries and source code are fed into the language models to localize design issues. The proposed approach is implemented as a Python-based tool that integrates the various components.

8.1.3 **LLM APIs**. We selected three below state-of-the-art LLMs:

ChatGPT (GPT-4o) [7]: GPT-4o is developed by OpenAI with enhanced reasoning and speed. It has an input token limit of 128K.

Gemini 1.0 pro [21]: Gemini 1.0 is an LLM built by Google with improved reasoning. It has an input token limit of 32K.

Claude 3.0 (Haiku) [4]: Claude 3.0 Haiku is a LLM from Anthropic. It has a higher input token limit, which is 200K.

8.1.4 **Metrics**. To evaluate LocalizeAgent's effectiveness in localizing design issues, we introduce the Exact Match @ K (EM@K) metric. This metric assesses the accuracy of the first K suggestions provided by our approach in correctly identifying the design issues.

Let $S = \{s_1, s_2, \ldots, s_K\}$ be the ordered set of the first K suggestions generated by our approach for a given design issue, and let g_t be the ground truth localization of the design issue. We define the Exact Match @ K metric as follows:

$$\text{EM@K} = \mathbb{1}(\exists s \in S : s = g_t) \tag{7}$$

where $\mathbb{1}(\cdot)$ is the indicator function that returns 1 if the condition inside the parentheses is true and 0 otherwise. In other words, EM@K is 1 if at least one of the first K suggestions exactly matches the ground truth localized design issues and 0 otherwise.

To compute the total Exact Match @ K (EM@K) across all design issues or refactoring types in the dataset, we use the following:

$$\frac{1}{N}\sum_{i=1}^{N} \text{EM@K}_i \tag{8}$$

where N is the dataset's total number of design issues or specific refactoring types. We compute EM@K for different values of K, such as EM@1, EM@5, and EM@10, to evaluate the average performance of the approach at different suggestion ranks.

8.2 RQ1: Effectiveness of LocalizeAgent on localizing design issues.

8.2.1 **RQ1(A): Localizing Single Design Issues.**

Experimental Setup: To evaluate LocalizeAgent's effectiveness in localizing design issues, we conduct experiments on the dataset described in § 8.1.1. Specifically, we focus on three main categories of design issues: modularity, information hiding, and complexity. For each category, we assess LocalizeAgent's performance using the Exact Match @ K (EM@K) defined in § 8.1.4. We compare LocalizeAgent's performance against three state-of-the-art LLMs: Claude 3, ChatGPT 4o, and Gemini 1.0. All APIs are run with temperature 0 and nucleus sampling (top p) as 1 to get a deterministic output for reproducibility purposes.

We also investigate LocalizeAgent's effectiveness across different refactoring types (Table 4). After filtering the dataset, we consider 4 refactoring types: parameterize variable, inline method, inline variable, and move method. For each refactoring type, we

Table 3: Results for Design Issue Localization per *Design Issue*

Design Issues		Claude 3			ChatGPT 4o			Gemini 1.0			Baseline-LLM		
		EM@1	EM@5	EM@10	EM@1	EM@5	EM@10	EM@1	EM@5	EM@10	EM@1	EM@5	EM@10
Complexity	Single	24.6	56.0	66.0	**28.5**	**59.3**	**66.2**	22.9	44.7	50.9	10.7	42.5	54.3
	Multi		48.6	55.4		**60.3**	**69.8**		45.7	56.0		37.9	47.4
Information Hiding	Single	30.3	52.8	58.4	**40.3**	**61.0**	**62.8**	32.9	47.6	48.9	16.9	48.5	53.7
	Multi		**58.9**	**66.1**		56.5	62.3		50.0	54.5		50.8	60.2
Modularity	Single	16.0	50.2	59.0	**24.5**	**54.0**	**62.9**	19.8	51.0	61.6	8.0	34.8	49.2
	Multi		40.3	48.2		**43.2**	**50.9**		34.8	41.9		32.7	45.8

Table 4: Results for Design Issue Localization per Refactoring Type

Refactoring Type		Claude 3			ChatGPT 4o			Gemini 1.0			Baseline-LLM		
		EM@1	EM@5	EM@10	EM@1	EM@5	EM@10	EM@1	EM@5	EM@10	EM@1	EM@5	EM@10
PV-Ref	Single	16.2	**51.4**	54.1	**24.3**	**51.4**	**56.8**	13.5	37.8	51.4	10.8	35.1	48.7
	Multi		**35.7**	50.0		**35.7**	35.7		**35.7**	**57.1**		28.6	**57.1**
IM-Ref	Single	21.2	54.9	**65.4**	**23.6**	**56.5**	63.6	18.1	40.7	48.1	8.4	39.6	51.9
	Multi		55.9	62.9		**59.2**	**67.1**		46.0	50.7		39.9	50.2
IV-Ref	Single	**47.7**	74.4	84.6	43.6	70.8	73.8	44.6	**78.5**	**88.7**	27.2	64.6	69.7
	Multi		44.4	54.0		49.2	60.3		49.2	**66.7**		**57.1**	**66.7**
MM-Ref	Single	19.3	54.0	62.6	**25.0**	**60.0**	**64.9**	22.6	51.7	60.5	8.8	36.7	49.7
	Multi		**47.6**	**54.9**		44.0	51.2		36.2	42.3		32.1	43.5

evaluate LocalizeAgent's localization accuracy and compare its performance against the above LLMs using the EM@K metric.

Baseline: As a baseline, we use GPT-4o API with a naive prompt without leveraging the multi-agent learning (named as Baseline-LLM in Tables 3 and 4). We selected the GPT-4o API as it showed a good balance between speed and performance when using it with our approach. The naive prompt asks the LLM to suggest a refactoring that could fix the design issues in the codebase without providing additional context. We also use temperature 0 and nucleus sampling (top p) as 1 to encourage a deterministic output.

Results: As seen in Tables 3 and 4, LocalizeAgent achieves high-performance gains over the baseline across all design issue categories and refactoring types. For complexity, information hiding, and modularity issues, LocalizeAgent with ChatGPT 4o outperforms the baseline by **166%, 138%, and 206%**, respectively, on EM@1. This improvement highlights the effectiveness of our multi-agent learning framework and context-aware prompting.

Table 4 shows LocalizeAgent's effectiveness for different refactoring types. Notably, LocalizeAgent demonstrates good performance on the IV-Ref refactoring type, with EM@1 scores surpassing the baseline by **60% to 75%** across all LLMs. This suggests that the proposed approach is particularly adept at handling inline variable refactorings, possibly due to the specific code context and patterns associated with this refactoring type.

The results also highlight some areas for improvement. The PV-Ref refactoring type exhibits lower performance compared to others, with EM@1 scores ranging from 13.5% to 24.3%. Despite this, LocalizeAgent still outperforms the baseline by **25% to 125%** for PV-Ref, indicating the potential for further improvement with additional context or refined prompting strategies.

Comparing the two tables, we observe an overlap in the relative performance of different LLMs. ChatGPT 4o consistently outperforms other models across both design issue categories and refactoring types. In contrast, Gemini 1.0 shows the lowest performance among the LLMs used with LocalizeAgent. However, even the least performing model with LocalizeAgent surpasses the baseline in all refactoring types, demonstrating its robustness. The baseline-LLM only exhibits better results than Gemini 1.0 in the PV-Ref refactoring type, while LocalizeAgent with Gemini 1.0 outperforms the baseline in the remaining refactoring types.

Case study evaluation. We evaluate our approach using the motivating example, which showcases PV-Ref type refactoring. Our experiment demonstrates improved performance by incorporating the relevant context, i.e., A_{CR} and A_{VU}. LocalizeAgent successfully identifies the function `initialiseAndInstallRepository` within the top 5 suggestions, indicating that the function requires parameterizing variables. In contrast, the naive prompt approach fails to localize the issue within the top 10 suggestions [2].

8.2.2 RQ1(B): Localizing multiple design issues.

Experimental Setup: To measure correctness, we check if all the ground truth suggestions are within the top K values. Since evaluating multiple suggestions with EM@1 would be infeasible, we only used EM@5 and EM@10 as evaluation metrics.

Baselines. We have a similar baseline as RQ1(A) 8.2.1.

Results: The experimental results show that localizing multiple design issues is more challenging than single issues, with consistently lower EM@5 scores. However, IM-Ref is an exception, showing comparative or even better accuracy for multi-issue localization. Interestingly, the baseline approach demonstrates a smaller performance gap between single and multiple issues compared to LocalizeAgent. While ChatGPT 4o dominated for single issues, Claude 3 showed better performance for multi-issue localization, with a 2.4 and 3.8 percentage point increase in the information hiding category. Despite the challenges, LocalizeAgent still outperforms the baseline in most design issues and refactoring types.

8.3 RQ2: Sensitivity Analysis.

Experimental Setup: To assess the sensitivity of LocalizeAgent to different API settings, we randomly sampled 37 codebases from each refactoring category in our dataset (PV-Ref, IM-Ref, IV-Ref, and MM-Ref). We then performed experiments by varying the temperature parameter. Temperature controls the LLM output's randomness. A higher temperature leads to more diverse and explorative suggestions, while a lower temperature results in more

Table 5: Ablation Study for Localizing Design Issue per Refactoring Type

Refactoring Type	Claude 3			ChatGPT 4o			Gemini 1.0		
	EM@1	EM@5	EM@10	EM@1	EM@5	EM@10	EM@1	EM@5	EM@10
PV-Ref	16.2	**51.4**	**54.1**	24.3	**51.4**	**56.8**	13.5	**37.8**	**51.4**
↪ w/o context	13.3	38.3	40.0	13.3	35.7	43.3	**21.7**	33.3	41.7
↪ w/o ranking	**18.9**	45.9	51.4	**27.0**	45.9	**56.8**	13.5	35.1	48.6
IM-Ref	21.2	**54.9**	**65.4**	23.6	**56.5**	**63.6**	18.1	40.7	48.1
↪ w/o context	23.4	54.6	61.0	21.7	45.6	51.0	**21.7**	**45.7**	**51.0**
↪ w/o ranking	**28.8**	54.6	64.2	**27.0**	51.3	62.9	17.4	36.5	43.3
IV-Ref	**47.7**	**74.4**	**84.6**	**43.6**	**70.8**	**73.8**	**44.6**	**78.5**	**88.7**
↪ w/o context	38.0	57.4	61.0	42.0	63.6	71.7	34.4	45.6	48.2
↪ w/o ranking	38.0	61.5	70.8	39.0	63.6	67.7	34.4	60.5	67.7
MM-Ref	19.3	**54.0**	**62.6**	25.0	60.0	64.9	22.6	51.7	60.5
↪w/o context	**28.4**	54.6	61.4	**29.1**	**61.5**	**66.8**	**28.5**	**54.6**	**61.3**
↪ w/o ranking	20.3	51.8	59.9	27.3	60.6	65.3	24.3	52.0	59.6

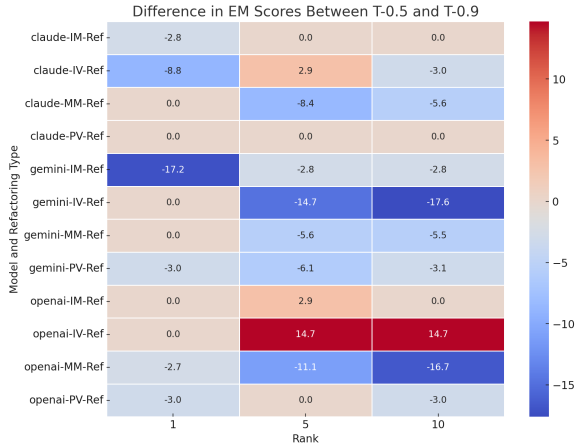


Figure 4: Heatmap result with different temperature values

focused and deterministic outputs. We evaluate LocalizeAgent using two temperature values: 0.5 (T-0.5) and 0.9 (T-0.9).

Results: Figure 4 shows the heatmap of the differences in EM scores between temperature settings of T-0.5 and T-0.9 for three LLMs and four refactoring types. The heatmap visually represents these differences; red indicates a performance decrease, and blue signifies a performance increase when changing from T-0.5 to T-0.9. Notably, an increase in temperature from T-0.5 to T-0.9 typically decreases the EM scores across most settings, suggesting that higher randomness might dilute the precision of the suggestions. For instance, the Gemini model under IM-Ref experienced a sharp decrease in EM@1, highlighting its particular sensitivity to increased temperature. Conversely, some refactoring types, such as IV-Ref with the Openai model, show a slight improvement with a higher temperature. This indicates that some refactoring types might benefit from the increased exploration, enabling the model to capture broader solution patterns. These findings illustrate the nuanced impact of temperature on model performance and underscore the necessity of tuning this parameter based on the specific context and objectives of the refactoring task.

8.4 RQ3: Contribution of each Component.

Experimental Setup: To assess the contribution of each component in LocalizeAgent, we conduct an ablation study by evaluating the performance under different settings, focusing on (1) the program analysis agent and (2) the ranking agent. First, we evaluate the effectiveness of LocalizeAgent without including any context (i.e., without $A_{FI/FO}$, A_{CR}, A_{VU}, A_{CC}). Next, we investigate the contribution of the ranking agent.

Results: Table 5 presents the results of an ablation study of LocalizeAgent. The results reveal that the impact of these components varies depending on the refactoring type and the LLM employed.

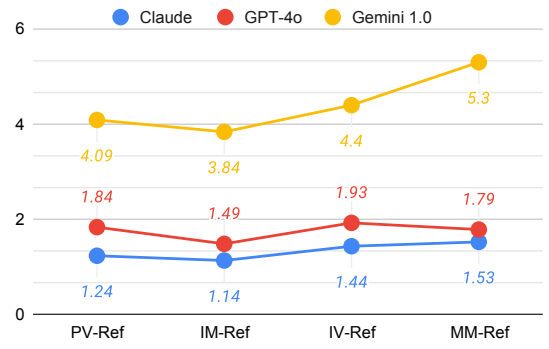
In the case of PV-Ref and IV-Ref, *context-aware prompting significantly improves performance across all LLMs.* For instance, the EM@1 scores drop by **2.9** to **11** percentage points for PV-Ref for Claude 3 and ChatGPT 4o. For IV-Ref without context, the results show a 1.6 to 10.2 percentage decrease across all models when run without using context. Additionally, the ranking component also shows a positive contribution, with EM@1 improvements ranging from **4.6** to **10.2** points for IV-Ref. ChatGPT 4o benefits the most from context and ranking, achieving the highest EM@1 scores of 24.3% and 43.6% for PV-Ref and IV-Ref, respectively.

In contrast, *the impact of context-aware prompting is less substantial* for IM-Ref and MM-Ref. For IM-Ref, Claude 3 and Gemini 1.0 show slight EM@1 improvements without context, while ChatGPT 4o's performance drops by 3.4 points, suggesting that the provided context might be excessive in some cases. For MM-Ref, which relies on the most context, i.e., $A_{FI/FO}, A_{CR}, A_{CC}$ summary, removing context results in the smallest impact on EM@1, with drops of 1 to 2.3 points. Our ranking shows a mixed impact, with improvements for Claude 3 and Gemini 1.0 but a slight decline for ChatGPT-4o.

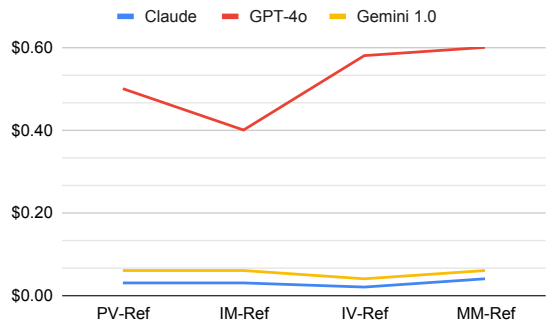
In brief, the effectiveness of context-aware prompting is closely tied to the complexity of the refactoring type and the LLM's inherent understanding of the associated design principles. The ranking component consistently improves performance across all refactoring types, emphasizing its role in refining the suggestions provided by the LLMs. ChatGPT 4o generally benefits the most from both context and ranking, while Gemini 1.0 and Claude 3 demonstrate competitive performance, particularly in scenarios where context-aware prompting has a limited impact.

8.5 RQ4: Time complexity and Cost of APIs.

Experimental Setup: We assessed the time complexity and cost of LLM APIs for each refactoring type by sampling 37 codebases randomly from each category. This approach maintains consistency in our dataset for fair comparisons. Time complexity was gauged by the end-to-end duration—from sending the request to receiving



(a) Time Complexity



(b) Cost of APIs

Figure 5: Time Complexity and Cost of LLM APIs per Refactoring Types

LLM-generated refactoring suggestions. Costs were calculated considering the number of API calls, input size, LLM outputs, and the pricing plans of each API provider. The total cost was then derived from these factors according to the respective pricing models.

Results: As Figure 5 shows, the results highlight differences in time complexity and cost among LLM APIs for design issue localization. Claude 3 (Haiku) displayed the lowest time complexity and API cost, proving the most efficient for quick feedback and budget-limited scenarios. ChatGPT 4o offered a balanced performance in speed but was costlier and suitable where trade-offs between speed and expense are acceptable. Gemini 1.0, while slowest, maintained a moderate cost, fitting for balancing multiple factors.

MM-Ref refactoring consistently showed higher time complexity and costs across all LLMs. This might be expected as it uses three different contexts ($A_{FI/FO}$, A_{CR},A_{CC}), increasing processing time. Yet, IM-Ref, also using three contexts, had lower time complexity, suggesting other factors influence processing speeds. IV-Ref, despite its smallest input size, had high time complexity, indicating that the refactoring type's inherent complexity and the LLM's efficiency in handling certain contexts significantly impact performance.

9 LIMITATIONS AND THREAT

Data leakage One potential limitation of our study is the risk of data leakage in the evaluation dataset. If the LLMs used in our approach were pre-trained on the same codebase or similar projects as those used in our evaluation, it could lead to an inflated performance metric. To mitigate this risk, we carefully selected a diverse set of real-world codebases from the refactoring dataset provided by Aniche *et al.* [1]. We also ran an ablation study on how each component in LocalizeAgent contributes to its performance.

Non-determinism of LLMs LLMs exhibit non-deterministic behavior, which can lead to variability in the generated refactoring suggestions. This non-determinism may affect the reproducibility and consistency of our results. To mitigate this issue, we employ a temperature setting of 0 for more deterministic refactoring suggestions. We presented the result with different settings in RQ2 8.3.

Rule-based filtering criteria We applied rule-based filtering criteria using PMD [18] to focus on specific refactoring types that successfully resolved design issues and contained the necessary ground truth. While these criteria help us evaluate the effectiveness of our approach in a controlled setting, they may limit the generalizability of our findings to other refactoring types or design issues. To address this, we plan to extend our approach to a broader range of refactoring types and design issues.

10 RELATED WORK

Several works have explored the use of LLMs for code understanding and generation tasks [6, 16, 23]. Recent works leverage emergent capabilities of LLMs, such as In-Context Learning (ICL) for code intelligence tasks [8, 9, 15, 17, 24], while others explore the use of agents and LLMs [5]. This work explores how static analysis-involved agents can supplement LLMs to localize refactorings that improve software design. Relevantly, Bouzenia et al. [5] equips an LLM tasked with automated program repair with "agents" that leverage external tools to explore and gather relevant data (exploring code base, testing patches, and planning goals) to supplement its input. Additionally, Zhang et al. [25] enables an LLM-based agent to explore a codebase via provided code search APIs to autonomously resolve a GitHub issue. Although Pomian et al. [19] proposes a technique to query and rank LLM-suggested extract method refactorings, we find a gap in the literature involving the leverage of agent-assisted LLMs towards multiple design issues.

11 CONCLUSION

This paper presents LocalizeAgent, a novel approach that leverages Large Language Models' (LLMs) code understanding to localize design issues in evolving codebases effectively. LocalizeAgent addresses the limitations of existing LLM-based approaches, such as their inability to analyze graph-based code analysis outputs and reason about complex code relationships, by leveraging multi-agent learning for collaborative refactoring, generating LLM-friendly natural language summaries of code metrics and dependencies and employing context-aware prompting tailored to specific refactoring types. Evaluated on real-world codebases, LocalizeAgent achieves a 206%, 166%, and 138% relative improvement in accuracy for modularity, complexity, and information hiding issues, respectively, compared to baseline LLM techniques.

REFERENCES

[1] Maurício Aniche, Erick Maziero, Rafael Durelli, and Vinicius H. S. Durelli. 2022. The Effectiveness of Supervised Machine Learning Algorithms in Predicting Software Refactoring. *IEEE Transactions on Software Engineering* 48, 4 (2022), 1432–1450. https://doi.org/10.1109/TSE.2020.3021736

[2] annonymous. 2024. ChatGPT Prompt of the Motivating Example. https://chatgpt.com/share/2986fa56-357d-4ea9-a821-9e8607389d9b.

[3] anonymous. 2024. LocalizeAgent Reproducibility Package. https://doi.org/10.5281/zenodo.11506893.

[4] AI Anthropic. 2024. The claude 3 model family: Opus, sonnet, haiku. *Claude-3 Model Card* (2024).

[5] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. arXiv:2403.17134 [cs.SE]

[6] Saikat Chakraborty, Toufique Ahmed, Yangruibo Ding, Premkumar T Devanbu, and Baishakhi Ray. 2022. NatGen: generative pre-training by "naturalizing" source code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 18–30.

[7] ChatGPT [n. d.]. OpenAI. https://openai.com/.

[8] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. 2024. Large Language Models are Few-Shot Summarizers: Multi-Intent Comment Generation via In-Context Learning. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 39, 13 pages. https://doi.org/10.1145/3597503.3608134

[9] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the Potential of ChatGPT in Automated Code Refinement: An Empirical Study. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 34, 13 pages. https://doi.org/10.1145/3597503.3623306

[10] Junda He, Christoph Treude, and David Lo. 2024. LLM-Based Multi-Agent Systems for Software Engineering: Vision and the Road Ahead. arXiv:2404.04834 [cs.SE]

[11] Ashish Hooda, Mihai Christodorescu, Miltos Allamanis, Aaron Wilson, Kassem Fawaz, and Somesh Jha. 2024. Do Large Code Models Understand Programming Concepts? A Black-box Approach. In *International Conference on Machine Learning*. PMLR.

[12] Maliheh Izadi, Roberta Gismondi, and Georgios Gousios. 2022. CodeFill: Multi-Token Code Completion by Jointly Learning from Structure and Naming Sequences. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. Association for Computing Machinery, 401–412. https://doi.org/10.1145/3510003.3510172

[13] JavaLang. 2024. Python Java parser and tools. https://pypi.org/project/javalang/.

[14] Lucas Layman, Gunnar Kudrjavets, and Nachiappan Nagappan. 2008. Iterative identification of fault-prone binaries using in-process metrics. In *Proceedings of the Second ACM-IEEE international symposium on Empirical software engineering and measurement*. 206–212.

[15] Ansong Ni, Miltiadis Allamanis, Arman Cohan, Yinlin Deng, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2024. NExT: Teaching Large Language Models to Reason about Code Execution. In *International Conference on Machine Learning*. PMLR.

[16] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. *arXiv preprint* (2022).

[17] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 82, 13 pages. https://doi.org/10.1145/3597503.3639226

[18] PMD. Accessed 2024. *PMD Static Analysis Tool*. https://docs.pmd-code.org/latest/pmd_rules_java_design.html

[19] Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Timofey Bryksin, and Danny Dig. 2024. Together We Go Further: LLMs and IDE Static Analysis for Extract Method Refactoring. arXiv:2401.15298 [cs.SE]

[20] Robert Schwanke, Lu Xiao, and Yuanfang Cai. 2013. Measuring architecture quality by structure plus history analysis. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 891–900.

[21] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

[22] Nikolaos Tsantalis, Matin Mansouri, Laleh M. Eshkevari, Davood Mazinanian, and Danny Dig. 2018. Accurate and Efficient Refactoring Detection in Commit History. In *Proceedings of the 40th International Conference on Software Engineering* (Gothenburg, Sweden) *(ICSE '18)*. ACM, New York, NY, USA, 483–494. https://doi.org/10.1145/3180155.3180206

[23] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D.Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. *arXiv preprint* (2023).

[24] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 126, 13 pages. https://doi.org/10.1145/3597503.3639121

[25] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. arXiv:2404.05427 [cs.SE]